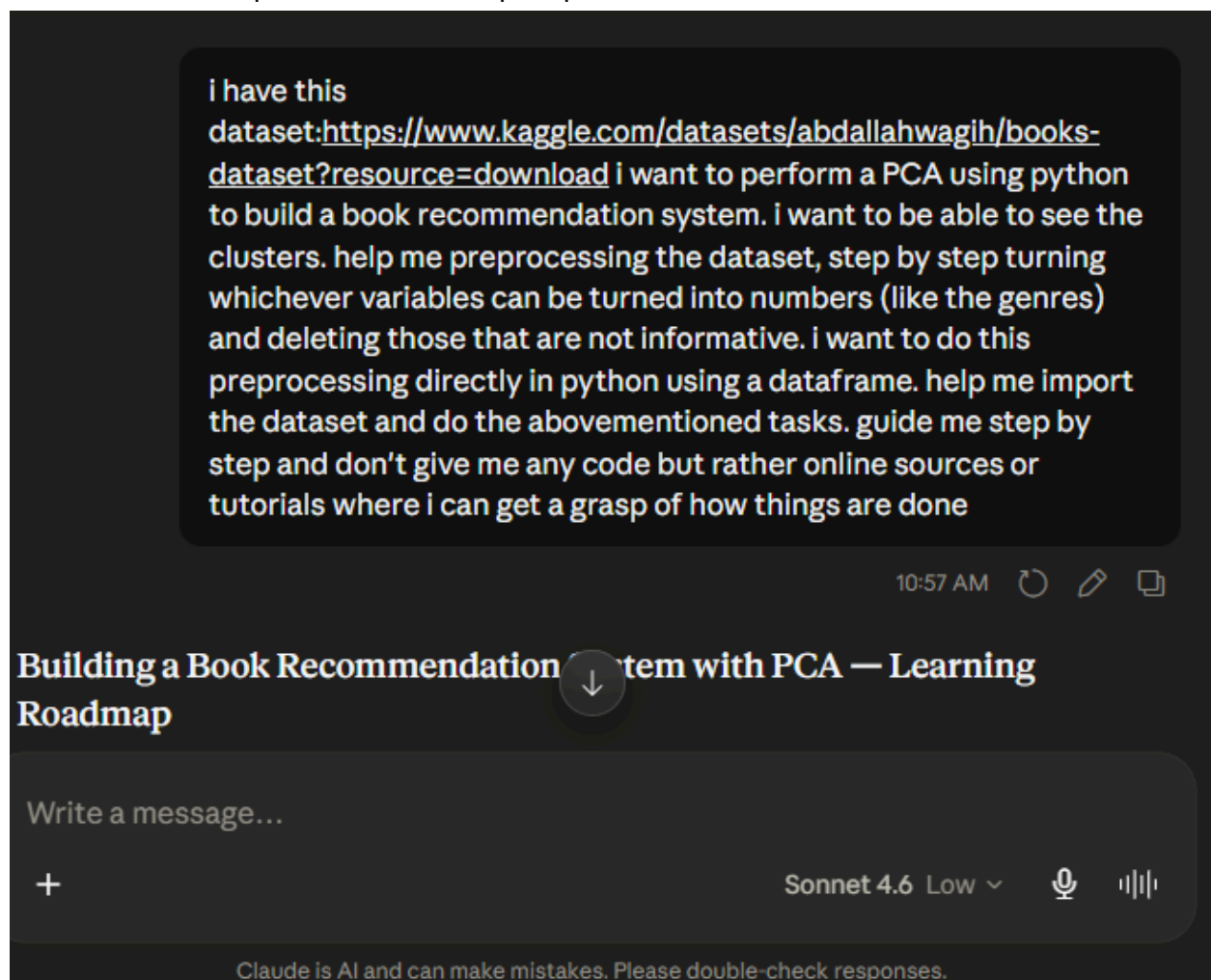


Declaración de uso de IA

- Herramientas utilizadas: Claude
- Actividades en las que se empleó IA Generativa: planteamiento y validación de la estructura de solución del equipo, apoyo con bugs y fallos del código. Generación de formato en blanco para la interfaz de usuario con la librería ipywidgets.
- Porcentaje estimado de uso: 18%
- Descripción de cómo se validó, corrigió o adaptó el contenido generado: se corroboró con la documentación de las librerías usadas para evitar alucinaciones y se buscaron otras alternativas de solución en sitios como GeeksforGeeks o stackoverflow. Asimismo se usó un notebook de Google Colab para consultar la documentación de ipywdgets: https://colab.research.google.com/github/jupyter-widgets/ipywidgets/blob/master/docs/source/examples/Widget%20List.ipynb#scrollTo=Updu_p-tnXhd
- Evidencia representativa de los prompts utilizados:



PROMPT: Quiero que me des un template general para la librería de ipywidget en un formato sencillo de entender y de aplicar por mi cuenta a un proyecto de recomendación de libros mediante K-NN.

Aquí está el template:

```
python
import ipywidgets as widgets
from IPython.display import display
```

— 1. WIDGETS

```
text_input = widgets.Text(
    placeholder='Texto de búsqueda...',
    layout=widgets.Layout(width='500px')
)
```

```
num_input = widgets.BoundedIntText(
    description='Seleccionar #:',
    min=1,
    layout=widgets.Layout(width='200px', display='none')
)
```

```
output = widgets.Output()
```

— 2. ESTADO COMPARTIDO

```
# Diccionario para pasar datos entre callbacks sin variables globales.
# Agregar aquí cualquier dato que más de un callback necesite leer o escribir.
```

```
estado = {
    'resultados': None, # resultados de la búsqueda actual (pd.Series u otro)
}
```

— 3. FUNCIONES AUXILIARES

```
# Separar la lógica de negocio de los callbacks hace el código más fácil de
# mantener y probar. Los callbacks solo orquestan; las funciones hacen el trabajo.
```

```
def buscar(texto):
    """Devuelve un pd.Series con los títulos que contienen `texto`."""
    return titles[titles.str.contains(texto, case=False, na=False)].drop_duplicates()
```

```
def recomendar(idx):
    """Devuelve las recomendaciones para el libro en posición `idx`."""
    return rec_system(idx) # rec_system definida en celda anterior
```

```
def mostrar_lista(resultados):
    """Imprime la lista numerada de resultados con autor."""
    print(f"Títulos encontrados ({len(resultados)}):\n")
    for i, titulo in enumerate(resultados.head(10), 1):
        autor = authors.iloc[resultados.index[i-1]]
        print(f" {i}. {titulo} - {autor}")

def mostrar_recomendaciones(idx, label):
    """Imprime las recomendaciones para un libro dado su índice."""
    recomendaciones = recomendar(idx)
    print(f"Recomendaciones para '{label}':\n")
    for i, r in enumerate(recomendaciones, 1):
        print(f" {i}. {r}")
```

— 4. CALLBACKS

```
def on_text_change(change):
    if change['type'] == 'change' and change['name'] == 'value':
        with output:
            output.clear_output()
            texto = change['new'].strip()
            num_input.layout.display = 'none'

            if len(texto) < 2:
                return

            resultados = buscar(texto)
            estado['resultados'] = resultados

            if len(resultados) == 0:
                print("No se encontraron resultados.")
                return

            if len(resultados) > 1:
                mostrar_lista(resultados)
                num_input.max = min(len(resultados), 10)
                num_input.value = 1
                num_input.layout.display = 'inline-flex'
                return

            # Exactamente un resultado: recomendar directamente
```

```

        idx = resultados.index[0]
        mostrar_recomendaciones(idx, f"{resultados.iloc[0]} - {authors.iloc[idx]}")

def on_num_change(change):
    if change['type'] == 'change' and change['name'] == 'value':
        with output:
            output.clear_output()
            resultados = estado['resultados']

            if resultados is None:
                return

            mostrar_lista(resultados)

            seleccion = change['new'] - 1
            idx = resultados.iloc[seleccion:seleccion+1].index[0]
            autor = authors.iloc[idx]
            titulo = resultados.iloc[seleccion]
            mostrar_recomendaciones(idx, f"{titulo} - {autor}")

```

— 5. REGISTRO Y DISPLAY

```

text_input.observe(on_text_change)
num_input.observe(on_num_change)

display(text_input, num_input, output)

```